OPIPE-99: Best Practice Summary

> **Series:** OPIPE — OpenPipeline Beyond Logs | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 04/25/2026

Definitive best practice settings for OpenPipeline beyond logs — spans, metrics, events, and cross-scope patterns. Each entry specifies the exact configuration.

Table of Contents

---

1. Multi-Scope Architecture

Practice	Recommended Setting/Value	Priority
One pipeline per source type per scope	Separate pipelines for each distinct data source	Critical

Practice	Recommended Setting/Value	Priority
Never leave >20% of volume in default pipeline	Target 0% in default for production environments	Critical
Route ordering: most specific first	Security/audit → application → infrastructure → API-ingested → default catch-all	Critical
Configure all six scopes independently	Logs, Spans, Metrics, Events, Business Events, Security Events — changes in one scope have zero effect on others	Recommended
Six-stage pipeline order in every scope	Routing → Masking → Filtering → Processing → Extraction → Storage	Critical
Default pipeline retention	`default_logs` : 14 days. `default_spans` : 7 days	Recommended
Audit default bucket percentage weekly	`countIf(dt.system.bucket == "default_logs")/count()` — CRITICAL if >80%, WARNING if >50%	Critical
Check pipeline count per scope	`countDistinct(dt.openpipeline.pipelines)` — minimum 3+ for logs, 2+ for spans	Critical

2. Processing Groups

Practice	Recommended Setting/Value	Priority
Use groups for different parsing within one pipeline	One group per log format (e.g., `java-logs` , `nodejs-logs`) with shared global processors outside	Recommended
Max 5-6 groups per pipeline	More than 6 = split into separate pipelines	Recommended
Never put bucket routing inside a group	Bucket assignment is pipeline-level, not per-group	Critical

Practice	Recommended Setting/Value	Priority
Make group matchers mutually exclusive	Unless multi-match is intentional — overlapping groups can set conflicting field values	Recommended
Use separate pipelines (not groups) when lifecycle differs	Different bucket, retention, or security context = separate pipeline	Critical
Groups for different processing, pipelines for different lifecycle	The fundamental decision rule	Critical

3. Span Filtering & Enrichment

Practice	Recommended Setting/Value	Priority
Drop health check spans	Drop: `span.name` matches `"GET /health*"` OR `"GET /ready*"` OR `"GET /livez*"`	Critical
Drop metrics scraping spans	Drop: `span.name` matches `"*/metrics*"` OR `"*/actuator/*"`	Critical
Drop OPTIONS preflight spans	Drop: `http.request.method == "OPTIONS"`	Recommended
Drop synthetic/heartbeat spans	Drop: `span.name` matches `"*synthetic*"` or `"*heartbeat*"`	Recommended
Quantify noise before configuring drops	Count health checks, scrapes, OPTIONS as % of total — typical noise: 30-60%	Recommended
Route spans to purpose buckets	Frontend: `frontend_spans` 14d. API: `api_spans` 35d. DB: `db_spans` 14d. Default: 7d	Recommended
Add `business.domain` enrichment	`service.name` starts with `"checkout"` → `business.domain = "commerce"`	Recommended
Add `environment` enrichment	`k8s.namespace.name` contains `"prod"` → `environment = "production"`	Recommended

Practice	Recommended Setting/Value	Priority
Pre-classify severity	`http.response.status_code >= 500` → `incident.severity = "high"`	Optional
Prefer metric extraction over event generation	Extraction: 1 data point/interval (low cost). Generation: 1 record/span (high cost)	Critical

4. Sampling-Aware Metrics & RED

Practice	Recommended Setting/Value	Priority
Enable sampling-aware for count metrics	`span.request_count`, `span.error_count` must weight by 1/sampling_ratio	Critical
Do NOT enable sampling-aware for duration	Averages and percentiles are statistically representative from sampled data	Recommended
Error rate ratios are naturally sampling-tolerant	Both numerator and denominator equally sampled — ratio is accurate	Recommended
Rate (RED)	Metric: `span.request_count`, count, `span.kind == "server"`, dims: `service.name`, `http.route`, sampling-aware: Yes	Critical
Errors (RED)	Metric: `span.error_count`, count, `span.kind == "server"` AND `status_code >= 500`, dims: `service.name`, `http.route`, `status_code`, sampling-aware: Yes	Critical
Duration (RED)	Metric: `span.duration`, value, `span.kind == "server"`, dims: `service.name`, `http.route`, sampling-aware: No	Critical
Validate against built-in metrics	Compare `span.request_count` vs `dt.service.request.count` — significant divergence = misconfiguration	Critical

5. Cardinality Management

Practice	Recommended Setting/Value	Priority
Never use `trace.id`, `span.id`, or `content` as dimensions	Unique per record = unbounded cardinality = metric dropped	Critical
Never use `http.url` as dimension	Use `http.route` (pattern) instead of `http.url` (instance with query params)	Critical
Never use `k8s.pod.name` without normalization	Use `k8s.deployment.name` or strip replica suffix	Critical
Remove `http.url` when `http.route` exists	`fieldsRemove` processor in Processing stage	Recommended
Remove `http.request.header.*` from spans	High-cardinality, rarely needed post-ingestion	Recommended
Normalize URL paths to patterns	`/users/12345/orders` → `/users/{id}/orders` via DPL parse	Critical
Group status codes into classes	`200/201/204` → `2xx`, `400/401/403` → `4xx`, `500/502/503` → `5xx`	Recommended
Cardinality budget: SLO metrics	<1,000 series, max 2-3 dimensions	Critical
Cardinality budget: operational metrics	<10,000 series, max 3-4 dimensions	Critical
Cardinality budget: exploratory metrics	<50,000 series, max 4-5 dimensions	Critical
Run cardinality estimation before extraction	`countDistinct()` across proposed dimensions on 1h data — reject if >100,000	Critical
Bucket continuous values into tiers	Duration: <100ms=fast, 100-500ms=normal, 500ms-2s=slow, >2s=very_slow	Recommended

Practice	Recommended Setting/Value	Priority
Hashing protects PII but does NOT reduce cardinality	100K emails → 100K hashes. Use removal/grouping to reduce.	Recommended

6. Security Context

Practice	Recommended Setting/Value	Priority
Set `dt.security_context` on every scope with sensitive data	Field enrichment processors in Logs, Spans, Metrics, Events, Bizevents	Critical
Extension metrics: set explicitly	Extensions 2.0 do NOT inherit security context — add via Metrics scope OpenPipeline enrichment on `metric.key` prefix	Critical
Use MATCH operator for array values	`=`, `STARTSWITH`, `IN` always return false for array-valued security contexts	Critical
Business events: by event type	Payment events → `"finance-team"`, signup → `"marketing-team"`, default → `"product-team"`	Recommended
Spans: by namespace or service	`k8s.namespace.name` matches `"team-alpha-*"` → `"team-alpha"`	Recommended

7. Business & Security Events

Practice	Recommended Setting/Value	Priority
Enrich business events with channel	`event.provider == "mobile-app"` → `channel = "mobile"`	Recommended
Extract conversion funnel metrics	Separate count per stage: views → adds → checkouts → purchases	Recommended

Practice	Recommended Setting/Value	Priority
Extract revenue with sum aggregation	`bizevent.purchase_revenue`, sum(`order.total`), dims: `provider`, `channel`	Recommended
Business event retention	90-365 days depending on reporting needs	Recommended
Never drop security events	No drop rules on any security pipeline	Critical
Never sample security events	Every individual event is potentially significant	Critical
Security event retention	Vulnerabilities/attacks/audit: 365 days. Compliance: 2555 days (7 years)	Critical
Align to compliance frameworks	SOC 2: 1 year. HIPAA: 6 years. PCI-DSS: 1 year. GDPR: as needed	Critical
Security context on all security pipelines	`dt.security_context = "security-team"`	Critical

8. Cross-Scope Patterns

Practice	Recommended Setting/Value	Priority
Primary correlation key: `dt.entity.service`	Ensure present on logs, spans, metrics, events — the single most important cross-scope join field	Critical
Enrich logs with `service.name` if missing	Derive from `dt.entity.service` via entity lookup	Recommended
Normalize `k8s.deployment.name` on spans	Strip replica suffix from `k8s.pod.name` if deployment name missing	Recommended
Extract same metric from two scopes for validation	`log.error_count` from logs + `span.error_count` from spans — divergence = pipeline issue	Recommended

Practice	Recommended Setting/Value	Priority
Name related buckets with common prefix	`checkout_logs`, `checkout_spans`, `checkout_events` — enables wildcard IAM	Recommended
Expire related buckets in logical sequence	Spans first (14d), logs (35d), events (35d), business events (90d)	Recommended
Use DQL for cross-scope correlation	OpenPipeline cannot join across scopes — use DQL `lookup` and `append`	Critical

9. Ingestion vs Query-Time

Practice	Recommended Setting/Value	Priority
Mask at ingestion — no exceptions	PII must be redacted before storage	Critical
Drop noise at ingestion	Debug, health checks, synthetic — never store and filter later	Critical
Set security context at ingestion only	Cannot be added retroactively to stored data	Critical
Extract metrics at ingestion only	Cannot create metrics from historical data	Critical
Route to buckets at ingestion only	Bucket assignment is permanent — cannot move data after storage	Critical
Keep raw for exploratory analysis	If you don't know what you're looking for, store raw and DQL at query time	Recommended
Decision order	(1) Sensitive? Mask. (2) Noise? Drop. (3) Access control? Security context. (4) Need metric? Extract. (5) Everything else? Store raw.	Critical

10. Production Readiness

Practice	Recommended Setting/Value	Priority
No overlapping routing conditions	Audit conditions for overlap — first match wins, order-dependent behavior is a bug	Critical
Drop rules before extraction rules	Filter noise before creating metrics — otherwise you extract from noise	Critical
Volume monitoring on every pipeline	`makeTimeseries count(), by: {dt.openpipeline.pipelines}, interval:1h` — alert on spikes/drops	Critical
Document rollback plan before changes	Record previous config — OpenPipeline cannot reprocess historical data	Critical
Metric naming convention	`scope.metric_name` (e.g., `span.request_count`, `log.error_count`)	Recommended
Test cascade chains end-to-end	Span → event → metric → SLO — failures propagate silently	Critical

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.